GPT-6 Astra and Fable 5.1

VulcanBench-SWE v4 | Five effort levels | September 2026

Abstract

Across 230 runs on 23 matched tasks, total scores span 90.97% to 92.90%. Astra in Codex has lower mean runtime and estimated API cost at every matched effort than Fable 5.1 in Claude Code with disclosed fallbacks. The highest observed total score is Fable at max (92.90%); Astra peaks at extra-high (92.39%). These are descriptive results from one attempt per task and effort, not a statistically established overall winner.

Total score combines functional correctness (50%), automated quality (15%), security (15%) and retrospective Code quality review (20%). Time and cost are separate. n=23 for every model/effort row; all ten rows use the same tasks and scoring protocol.

Table 1. Performance by effort

Model	Effort	Total	SE (pts)	Code quality	Min/task	Tokens/task	API $/task
Astra	Low	91.60%	0.44	81.89	4.2	0.89M	$1.72
Astra	Medium	91.73%	0.23	83.04	3.8	0.68M	$1.48
Astra	High	91.59%	0.18	85.18	4.9	0.77M	$1.71
Astra	Extra-High	92.39%	0.17	86.17	8.1	0.93M	$2.30
Astra	Max	92.25%	0.21	86.54	10.3	0.95M	$2.57
Fable*	Low	91.04%	1.31	80.72	26.9	4.38M	$7.96
Fable*	Medium	91.56%	1.09	82.43	31.7	6.97M	$9.19
Fable*	High	90.97%	0.93	83.81	28.8	5.49M	$9.70
Fable*	Extra-High	91.53%	0.93	84.96	38.8	11.44M	$14.49
Fable*	Max	92.90%	0.49	85.36	27.1	3.82M	$9.06

SE is one sample standard error across tasks, not judge uncertainty or a significance test. Code quality is out of 100. Runtime, raw tokens and costs are means per task. API estimates use a frozen September 6, 2026 rate snapshot, not subscription charges.

Findings

Effort buys different tradeoffs. Astra medium reaches 91.73% at 3.8 minutes and $1.48 per task. Extra-high adds 0.66 score points at 8.1 minutes and $2.30. For Fable, max scores 1.86 points above low at nearly the same mean runtime, but costs $9.06 rather than $7.96.

Code quality rises more than total score. Low-to-max Code quality gains are 4.65 points for both models, while automated quality and security also move. More effort is not a uniform improvement across all four factors.

Costs separate these runs more than scores. Full-sweep estimates are $225.04 for Astra and $1,159.10 for Fable. Astra's conservative long-context estimate is $419.42. Missing request-level context sizes limit the precision of that comparison.

Score components and reviewer calibration

Partial-credit functional scores and all non-functional factors contribute to every total. A perfect functional result is not a 100% total score. The fixed 20% review weight is a benchmark policy, not an empirically calibrated measure of production value.

Table 2. Four factors, mean score out of 100

Model	Effort	Functional	Auto quality	Security	Code quality
Astra	Low	99.71	81.30	87.83	81.89
Astra	Medium	100.00	81.60	85.87	83.04
Astra	High	100.00	81.05	82.61	85.18
Astra	Extra-High	100.00	81.62	86.09	86.17
Astra	Max	100.00	80.87	85.43	86.54
Fable*	Low	96.12	82.59	96.30	80.72
Fable*	Medium	97.58	83.28	91.96	82.43
Fable*	High	98.46	82.63	83.91	83.81
Fable*	Extra-High	99.13	83.22	83.26	84.96
Fable*	Max	100.00	82.16	90.00	85.36

Functional correctness carries half the weight. Automated quality uses static quality indicators; security uses static analysis. Neither guarantees production readiness or freedom from vulnerabilities. Code quality is a separate model-reviewed measure.

Table 3. Reviewer-panel means, score out of 100

Solver	Effort	Astra reviewer	Claude reviewer	Equal-panel mean
Astra	Low	89.17	74.61	81.89
Astra	Medium	90.20	75.88	83.04
Astra	High	91.52	78.84	85.18
Astra	Extra-High	92.36	79.98	86.17
Astra	Max	92.52	80.55	86.54
Fable*	Low	85.91	75.52	80.72
Fable*	Medium	87.38	77.49	82.43
Fable*	High	87.72	79.90	83.81
Fable*	Extra-High	89.62	80.29	84.96
Fable*	Max	89.25	81.48	85.36

Claude's panel scores are lower for both solver populations. This is a systematic calibration difference, not evidence that either panel is objective. Equal weighting reduces reliance on one panel but does not remove self-preference, shared bias or rubric sensitivity.

Each panel averages correctness, readability and maintainability personas, rounds that 0-to-1 mean to four decimals, and then receives half the Code quality weight. The public record includes all selected numerical ratings and the exact general review rubric.

API-equivalent cost and elapsed time

Costs describe hypothetical Standard API inference charges for exposed solver usage. They are not subscription cash charges. Post-hoc judges and local infrastructure are excluded; no Batch, Flex, Fast or regional modifiers are assumed.

Table 4. Cost of each 23-task effort sweep, USD

Effort	Astra	Astra upper bound	Fable with fallbacks
Low	$39.60	$75.53	$183.17
Medium	$34.07	$64.19	$211.28
High	$39.35	$74.00	$223.14
Extra-High	$52.95	$97.84	$333.17
Max	$59.07	$107.86	$208.35
All efforts	$225.04	$419.42	$1,159.10

Astra input already contains cached input and its output already contains reasoning tokens. The central estimate assumes requests at or below 272,000 input tokens. Individual request sizes are absent; cumulative task tokens are not treated as a single request. The upper scenario applies long-context rates to all usage in runs whose cumulative input exceeds that threshold. It is a sensitivity bound, not a confidence interval.

Table 5. Frozen Standard rates, USD per million tokens

Model	Input	Cache read	Write 5m	Write 1h	Output
Astra	10.00	1.00	12.50*	n/a	50.00
Fable 5.1	10.00	0.25	12.50	20.00	50.00
Opus 5 / 4.8	5.00	0.50	6.25	10.00	25.00
Haiku 4.5	1.00	0.10	1.25	2.00	5.00

*Astra cache-write rate; zero cache writes were reported. Its long-context rates multiply input/cache prices by 2 and output by 1.5. Claude cache durations are priced separately. Sources: OpenAI pricing and Anthropic pricing, checked September 6, 2026.

Fable costs use final cumulative modelUsage receipts per session, not the sum of repeated cumulative snapshots. Actual Fable, Opus and Haiku rates are used for 252 model-level entries. Of these, 251 independently reconcile to cache-duration evidence; one internal Opus 5 receipt ($0.40398125) lacks that trace and matches five-minute cache pricing.

Table 6. Solver time, hours

Model	Summed solver time	First-start to last-finish
Astra	12.00	12.01
Fable*	58.75	80.32

Summed solver time is 70.75 hours across both sweeps. Calendar spans include gaps and overlap across models, so they must not be added. These figures exclude post-hoc judging. Run timestamps and exact seconds are available in the GitHub record.

Method, audit scope and limitations

Matched tasks and recorded submissions

Both sweeps use the same 23 behavioral-reconstruction tasks and five requested effort settings, with one original attempt per task and effort and a 10-hour per-task limit. The agent repairs a Python replacement for an opaque C-built binary. This is one implementation language, not evidence of broad multi-language coverage.

Astra used Codex CLI 0.153.4. Fable used Claude Code 2.1.259 through 2.1.261. Effort labels are harness-specific controls, not equal compute budgets. Eleven Fable runs include Opus fallbacks after refusal events: low 1, medium 3, high 2, extra-high 3 and max 2. Those runs remain included and labeled, not silently described as pure Fable results.

Retrospective review with two panels

Each saved solution receives three Astra and three Claude ratings at medium effort. Reviewers receive the issue, full saved patch and verifier outcome; solver identity and effort labels are omitted. Tools are disabled and sessions are separate. Review changes neither the solution nor the original functional grade. Claude's reviewer is Opus 5, with 10 selected ratings using Opus 4.8; this is not a Fable reviewer panel.

Solver / reviewer	Calls	Selected	Excluded	Fallback ratings
Astra / Astra	345	345	0	0
Astra / Claude	347	345	2	10
Fable* / Astra	345	345	0	0
Fable* / Claude	349	345	4	0

All 1,380 selected ratings are retained from 1,386 calls. Two Fable/Claude ratings used deterministic format recovery. For High QueueCore readability, the first rating (70) is selected and the later retry (72) is excluded. Selection was not based on a higher score.

What the checks do and do not establish

The export verified saved hashes for all 230 solver streams and 1,386 reviewer streams. Public checks recompute scores, panel means, task aggregates, standard errors and cost totals. Hash consistency does not prove absence of prohibited access, cheating or training-data overlap. Astra's stream reports the requested model only, not a returned model identity.

One attempt per cell cannot establish run-to-run reliability. Task SE is not reviewer uncertainty or a test of significance. Reviewer calibration differs, static analysis is imperfect, CLI versions vary, and native auxiliary accounting differs across providers. Scores are not directly comparable with the older Eval Suite 3.

Public evidence and follow-up

Open the immutable GitHub run record: 230 run records, selected ratings, all reviewer-call dispositions, rate assumptions, source hashes and verification scripts. Raw task prompts, patches, rationales, hidden graders and trajectories are withheld; the bundle does not independently prove those contents.

Use these results to shortlist an effort setting, then validate it on your own workload. A stronger follow-up would repeat paired runs with fixed CLI versions, returned model identities, per-request token accounting and independently audited confinement.

Appendix. GPT-6 Astra

Total score by task and effort (%)

Every cell combines the same four weighted factors, including 20% Code quality. These are total scores, not binary pass counts. Full-precision component scores, timing, token usage and cost records are available on GitHub.

Task	Low	Medium	High	Extra-high	Max
blendcore	90.57	90.86	91.57	91.84	91.99
cellarcore	93.70	92.32	90.65	92.61	91.00
codeccore	94.26	91.66	91.42	92.84	93.31
depotcore	93.48	93.71	92.42	91.98	90.95
freightcore	91.70	89.99	91.79	91.80	92.89
granarycore	89.41	91.08	90.91	92.39	91.97
hedgecore	94.70	92.84	91.54	91.15	93.10
lodgecore	91.88	89.86	91.62	91.90	90.91
matchcore	90.76	91.61	90.23	90.87	91.65
metercore	90.44	91.00	91.05	91.44	91.81
pacecore	90.49	90.57	93.29	93.35	93.22
paddockcore	84.31	91.63	90.10	92.38	90.14
payrollcore	90.81	91.41	91.62	92.72	93.07
qlite	92.70	92.38	90.94	92.86	93.24
queuecore	92.08	92.81	91.41	92.71	92.83
quotacore	92.47	91.33	91.49	93.64	92.92
reflowcore	91.62	91.85	91.59	93.74	92.01
schedcore	91.86	92.29	92.79	92.85	93.05
settlecore	91.04	92.00	90.91	92.85	90.52
snapcore	91.59	90.28	91.70	93.20	93.36
stampcore	90.07	93.22	92.81	91.43	93.32
tallycore	93.91	94.09	91.52	92.90	93.04
vaultcore	92.99	90.95	93.11	91.53	91.54

PaddockCore at low has a 93.33% functional component; it is not a perfect functional result. The total score still incorporates its quality, security and reviewer values. A strong total does not erase partial correctness.

Source: public runs.json. Display rounding only; calculations use full precision. Each column contains the same 23 tasks and one attempt per task.

Appendix. Fable 5.1 with fallbacks

Total score by task and effort (%)

Every cell combines the same four weighted factors, including 20% Code quality. These are total scores, not binary pass counts. Full-precision component scores, timing, token usage and cost records are available on GitHub.

Task	Low	Medium	High	Extra-high	Max
blendcore	94.44	91.93	94.92	94.80	95.18
cellarcore	93.73	93.35	87.69	89.71	93.18*
codeccore	75.25	92.59	95.04	95.16	94.75
depotcore	92.49	82.53*	86.75	92.19*	93.12
freightcore	93.95	95.12	95.06	95.20	94.99
granarycore	78.55*	77.37*	92.22	90.38*	90.51
hedgecore	95.12	94.83	87.93	79.93	94.17
lodgecore	92.60	94.68	85.82*	93.29	93.18
matchcore	93.94	95.17	94.86	91.88	87.26
metercore	94.92	94.71	95.29	94.35	95.03
pacecore	94.69	94.43	95.01	88.74	94.52
paddockcore	92.31	91.67	92.53*	80.02*	88.78
payrollcore	94.71	89.93	87.87	87.19	91.44
qlite	88.94	85.12	95.02	94.69	94.30
queuecore	77.75	94.92	90.13	94.56	94.27*
quotacore	91.20	95.08	87.99	88.17	94.78
reflowcore	95.20	94.73	94.69	88.97	95.04
schedcore	94.73	93.48	90.63	93.19	91.00
settlecore	80.93	80.45	81.16	94.56	94.23
snapcore	94.88	95.23	81.60	94.75	88.49
stampcore	94.70	94.88*	94.50	94.63	94.91
tallycore	94.19	94.61	90.04	93.69	92.19
vaultcore	94.62	89.10	95.59	95.13	91.28

*Includes solver fallback usage. The label marks the original run identity and does not change its score. Partial functional results remain partial in the public component records; no missing factor is imputed.

Source: public runs.json. Display rounding only; calculations use full precision. Each column contains the same 23 tasks and one attempt per task.

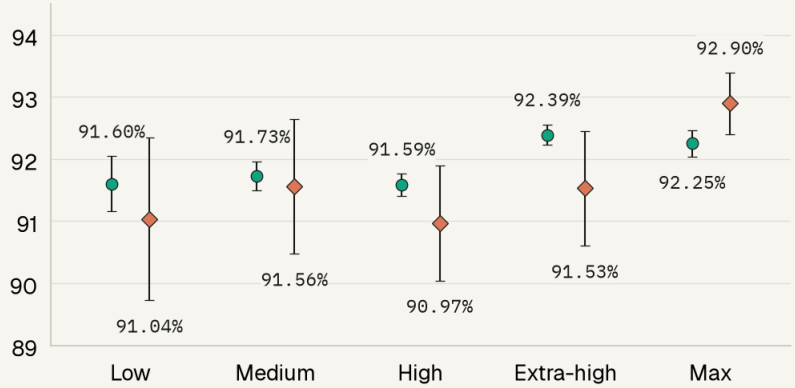
VulcanBench

VulcanBench-SWE v4 / September 2026

● GPT-6 Astra · Codex ◆ Fable 5.1 with fallbacks* · Claude Code n=23 at every effort

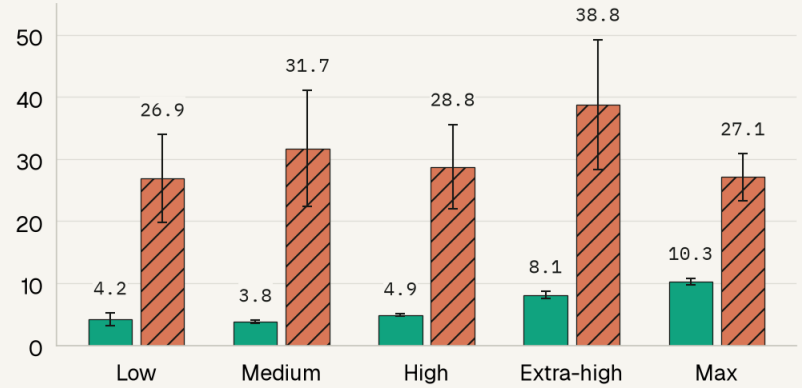
Total score

% · all four metrics · focused scale



Time per task

Mean minutes · lower is better



Astra Medium: total 91.73%; API $1.48/task.

Astra	Code quality	Total score	Tokens/task	API $/task†
Low	81.89 ± 0.65	91.60%	0.89M	$1.72
Medium	83.04 ± 0.44	91.73%	0.68M	$1.48
High	85.18 ± 0.39	91.59%	0.77M	$1.71
Extra-high	86.17 ± 0.37	92.39%	0.93M	$2.30
Max	86.54 ± 0.30	92.25%	0.95M	$2.57

Fable Low: total 91.04%; API $7.96/task.

Fable*	Code quality	Total score	Tokens/task	API $/task†
Low	80.72 ± 1.82	91.04%	4.38M	$7.96
Medium	82.43 ± 1.66	91.56%	6.97M	$9.19
High	83.81 ± 1.23	90.97%	5.49M	$9.70
Extra-high	84.96 ± 1.28	91.53%	11.44M	$14.49
Max	85.36 ± 0.40	92.90%	3.82M	$9.06

Weights: 50% functional + 15% automated quality + 15% security + 20% Code quality

Code quality: equal Astra + Claude panels. Whiskers and ±: 1 task SE, not judge uncertainty or significance.

†USD API-equivalent estimates, not subscription charges. Official rates checked Sep 6, 2026; cache-aware; judging excluded.

Astra assumes requests ≤272k input tokens. Request sizes are unlogged: conservative sweep upper bound $419, still 64% lower.

*11/115 Fable runs include Opus fallback usage; auxiliary calls priced too. 10/690 Claude ratings use Opus 4.8. LLM bias remains possible.

Code quality /100 · Raw tokens include cache · 230 runs · 23 Python replacements for C-built binaries · Effort labels are harness-specific